

# Quantum Computing

Advanced Reference & Teaching Handout

*Classical vs Quantum — Complexity Classes — Algorithms — Implementation*

**Machiraju Karthikeya**

BTech CSE — Quantum Computing Researcher

PES University, Bangalore

November 27, 2025

This comprehensive handout provides a complete mathematical treatment of quantum computing, designed for both teaching and reference. The document covers foundational concepts (classical vs quantum computation), complexity classes (P, NP, BPP, BQP), and all major quantum algorithms with detailed derivations, worked examples, and practical implementations using Qiskit. Each topic includes conceptual explanations, rigorous mathematics, multiple solved examples, practice problems at varying difficulty levels, and complete Python code implementations. This handout serves as both a learning resource for students and a reference guide for instructors.

**Coverage:** Classical computation models, quantum bits and superposition, complexity theory, quantum oracles, Deutsch-Jozsa algorithm, Bernstein-Vazirani algorithm, Simon's algorithm, Quantum Phase Estimation, Shor's factoring algorithm, Grover's search algorithm, Grover-Radhakrishnan partial search, Quantum Fourier Transform, Qiskit implementations, and photonic implementations on silicon photonics.

**Structure:** 15+ chapters, 150+ mathematical derivations, 100+ worked examples with complete solutions, 100+ practice problems (tiered by difficulty), 30+ complete Qiskit code implementations, extensive conceptual explanations with geometric intuitions.

# Contents

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| <b>I</b>  | <b>Computational Foundations &amp; Complexity Theory</b>             | <b>5</b>  |
| <b>1</b>  | <b>Classical vs Quantum Computers: Complete Comparative Analysis</b> | <b>6</b>  |
| 1.1       | Learning Objectives . . . . .                                        | 6         |
| 1.2       | 1.1 Classical Computation: The Turing Machine Model . . . . .        | 7         |
| 1.3       | 1.2 Classical Resources: Time and Space Complexity . . . . .         | 8         |
| 1.4       | 1.3 Quantum Bits and Superposition . . . . .                         | 9         |
| 1.5       | 1.4 Multi-Qubit Systems and Quantum Parallelism . . . . .            | 10        |
| 1.6       | 1.5 Classical vs Quantum: Side-by-Side Comparison . . . . .          | 11        |
| <b>2</b>  | <b>Complexity Classes: P, NP, BPP, BQP</b>                           | <b>12</b> |
| 2.1       | Learning Objectives . . . . .                                        | 12        |
| 2.2       | 2.1 P: Polynomial-Time Decidable Languages . . . . .                 | 12        |
| 2.3       | 2.2 NP: Nondeterministic Polynomial-Time . . . . .                   | 13        |
| 2.4       | 2.3 NP-Completeness . . . . .                                        | 15        |
| 2.5       | 2.4 BPP: Probabilistic Polynomial-Time . . . . .                     | 15        |
| 2.6       | 2.5 BQP: Quantum Polynomial-Time . . . . .                           | 17        |
| 2.7       | 2.6 Complexity Class Relationships . . . . .                         | 17        |
| <b>II</b> | <b>Quantum Algorithms &amp; Oracles</b>                              | <b>19</b> |
| <b>3</b>  | <b>Quantum Oracles and Phase Kickback</b>                            | <b>20</b> |
| 3.1       | Learning Objectives . . . . .                                        | 20        |
| 3.2       | 3.1 Quantum Oracle Definition . . . . .                              | 20        |
| 3.3       | 3.2 Phase Kickback Mechanism . . . . .                               | 21        |
| <b>4</b>  | <b>Deutsch-Jozsa Algorithm: Exponential Speedup</b>                  | <b>24</b> |
| 4.1       | Learning Objectives . . . . .                                        | 24        |
| 4.2       | 4.1 Problem Statement . . . . .                                      | 24        |
| 4.3       | 4.2 Quantum Algorithm: Step-by-Step Derivation . . . . .             | 25        |
| 4.4       | 4.3 Correctness Analysis . . . . .                                   | 27        |
| 4.5       | 4.4 Complexity and Speedup . . . . .                                 | 27        |
| 4.6       | 4.5 Practice Problems . . . . .                                      | 30        |
| <b>5</b>  | <b>Bernstein-Vazirani Algorithm</b>                                  | <b>31</b> |
| 5.1       | Learning Objectives . . . . .                                        | 31        |
| 5.2       | 5.1 Problem Statement . . . . .                                      | 31        |
| 5.3       | 5.2 Algorithm . . . . .                                              | 32        |
| <b>6</b>  | <b>Simon's Algorithm</b>                                             | <b>34</b> |
| 6.1       | 5.1 Problem Statement . . . . .                                      | 34        |
| 6.2       | 5.2 Simon's Algorithm . . . . .                                      | 35        |

|            |                                                         |           |
|------------|---------------------------------------------------------|-----------|
| <b>7</b>   | <b>Quantum Phase Estimation</b>                         | <b>36</b> |
| 7.1        | Learning Objectives . . . . .                           | 36        |
| 7.2        | 7.1 Problem . . . . .                                   | 36        |
| 7.3        | 7.2 QPE Algorithm . . . . .                             | 37        |
| <b>8</b>   | <b>Shor's Algorithm: Factoring</b>                      | <b>38</b> |
| 8.1        | Learning Objectives . . . . .                           | 38        |
| 8.2        | 8.1 Factorization Problem . . . . .                     | 38        |
| 8.3        | 8.2 Order-Finding Problem . . . . .                     | 39        |
| 8.4        | 8.3 Shor's Quantum Solution . . . . .                   | 41        |
| <b>9</b>   | <b>Grover's Search Algorithm</b>                        | <b>42</b> |
| 9.1        | Learning Objectives . . . . .                           | 42        |
| 9.2        | 9.1 Problem Statement . . . . .                         | 42        |
| 9.3        | 9.2 Geometric Analysis in 2D . . . . .                  | 43        |
| 9.4        | 9.3 Grover Operator . . . . .                           | 44        |
| 9.5        | 9.4 Optimal Iterations . . . . .                        | 44        |
| <b>10</b>  | <b>Grover-Radhakrishnan: Partial Grover's Search</b>    | <b>46</b> |
| 10.1       | 10.1 Multiple Marked Items . . . . .                    | 46        |
| 10.2       | 10.2 Modified Algorithm . . . . .                       | 47        |
| <b>11</b>  | <b>Quantum Fourier Transform</b>                        | <b>48</b> |
| 11.1       | Learning Objectives . . . . .                           | 48        |
| 11.2       | 11.1 Classical vs Quantum . . . . .                     | 48        |
| 11.3       | 11.2 QFT Matrix Form . . . . .                          | 49        |
| 11.4       | 11.3 Efficient QFT Circuit . . . . .                    | 49        |
| 11.5       | 11.4 Gate Count Analysis . . . . .                      | 51        |
| <b>III</b> | <b>Implementation &amp; Applications</b>                | <b>52</b> |
| <b>12</b>  | <b>Qiskit Implementation: Complete Working Examples</b> | <b>53</b> |
| 12.1       | Learning Objectives . . . . .                           | 53        |
| 12.2       | 12.1 Deutsch-Jozsa Implementation . . . . .             | 55        |
| 12.3       | 12.2 Grover's Algorithm . . . . .                       | 57        |
| 12.4       | 12.3 Quantum Fourier Transform . . . . .                | 58        |
| <b>13</b>  | <b>Quantum Entanglement on Silicon Photonics</b>        | <b>59</b> |
| 13.1       | 13.1 Photonic Implementation . . . . .                  | 59        |
| 13.2       | 13.2 Experimental Results . . . . .                     | 61        |
| 13.3       | 13.3 3-Qubit QFT on Photonics . . . . .                 | 62        |
| <b>14</b>  | <b>Summary &amp; Key Takeaways</b>                      | <b>63</b> |
| 14.1       | 14.1 Algorithm Comparison . . . . .                     | 63        |
| 14.2       | 14.2 Key Principles of Quantum Algorithms . . . . .     | 63        |
| 14.3       | 14.3 Complexity Classes Summary . . . . .               | 64        |
| 14.4       | 14.4 Limitations and Open Problems . . . . .            | 64        |
| 14.5       | 14.5 Future Directions . . . . .                        | 64        |

**Part I**

**Computational Foundations &  
Complexity Theory**

# Chapter 1

## Classical vs Quantum Computers: Complete Comparative Analysis

### 1.1 Learning Objectives

By the end of this chapter, you will:

1. Understand the formal definition of Turing machines
2. Compare classical and quantum computational models
3. Grasp quantum superposition and its implications
4. Identify key resource differences between classical and quantum systems

## 1.2 1.1 Classical Computation: The Turing Machine Model

### Definition: Turing Machine

A Turing Machine is a theoretical computational model consisting of a 7-tuple  $M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$ :

- $Q$ : Finite set of states
- $\Sigma$ : Input alphabet (symbols to process)
- $\Gamma$ : Tape alphabet ( $\Sigma \subseteq \Gamma$ , includes blank symbol)
- $\delta$ : Transition function  $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$
- $q_0$ : Initial state
- $q_a$ : Accept/halt state
- $q_r$ : Reject/halt state

**Computation Model:** At each step  $t$ , the machine state is  $(q_t, p_t, \text{tape}_t)$  where:

- $q_t$  = current state
- $p_t$  = head position on tape
- $\text{tape}_t$  = current tape contents

**Transition Step:** Read symbol  $\gamma$  at position  $p_t$ . Using  $\delta(q_t, \gamma) = (q', \gamma', d)$ :

1. Write  $\gamma'$  at position  $p_t$
2. Move head in direction  $d \in \{L, R, S\}$  (Left/Right/Stay)
3. Move to state  $q'$

**Halting:** Stop when  $q = q_a$  (accept) or  $q = q_r$  (reject).

**Remember:** Remember: Deterministic Sequential Processing Classical computers process information sequentially. At each moment, the system is in exactly one state processing one input configuration. This is fundamentally different from quantum systems.

**Theorem:** Church-Turing Thesis

Every effectively computable function can be computed by a Turing Machine. Equivalently, all reasonable computational models (RAM machines, lambda calculus, circuit models) are computationally equivalent.

**Implications:**

1. Complexity classes (P, NP, EXP) are model-independent
2. Any computational device using classical physics operates at the level of Turing machines
3. Time complexity is a fundamental property of the problem, not the implementation

## 1.3 1.2 Classical Resources: Time and Space Complexity

**Definition:** Time Complexity Classes

For a Turing Machine  $M$  computing language  $L$ :

**Time Complexity:**  $T_M(n)$  = maximum number of steps  $M$  takes on any input of length  $n$ .

**Time Complexity Class:**

$$\text{TIME}(f(n)) = \{L : \exists \text{ TM } M, L(M) = L, T_M(n) = O(f(n))\}$$

**Key Complexity Classes:**

$$\mathbf{P} = \bigcup_{k=0}^{\infty} \text{TIME}(n^k) = \text{polynomial-time decidable} \quad (1.1)$$

$$\mathbf{EXP} = \bigcup_{k=0}^{\infty} \text{TIME}(2^{n^k}) = \text{exponential-time decidable} \quad (1.2)$$

**Example:** Classical Algorithm Complexity Examples

| Problem           | Algorithm    | Complexity               | Status                     |
|-------------------|--------------|--------------------------|----------------------------|
| Sorting           | Merge Sort   | $O(n \log n)$            | $\in \mathbf{P}$           |
| Multiplication    | Grade-school | $O(n^2)$                 | $\in \mathbf{P}$           |
| Primality Testing | AKS Test     | $O((\log n)^6)$          | $\in \mathbf{P}$           |
| Factorization     | GNFS         | $\exp(1.9(\ln N)^{1/3})$ | NOT known $\in \mathbf{P}$ |
| SAT (n vars)      | Brute force  | $O(2^n)$                 | NP-complete                |

**Note:** For an  $n$ -bit number  $N$ , we have  $N \approx 2^n$ . So  $O(2^n)$  is exponential in the number of bits, not the number itself.

## 1.4 1.3 Quantum Bits and Superposition

**Key Insight:** Quantum Bit (Qubit)

A quantum bit is the fundamental unit of quantum information. Unlike classical bits (either 0 or 1), a qubit can exist in a linear combination of both states simultaneously.

**Definition:** Qubit and Superposition

**Qubit State:** A normalized state in  $\mathbb{C}^2$ :

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1$$

**Interpretation:**

- $|\alpha|^2$  = probability of measuring 0
- $|\beta|^2$  = probability of measuring 1
- Phase  $\arg(\alpha)$  and  $\arg(\beta)$  encode quantum information inaccessible by measurement

**Bloch Sphere Representation:** Any qubit can be written as:

$$|\psi\rangle(\theta, \phi) = \cos(\theta/2) |0\rangle + e^{i\phi} \sin(\theta/2) |1\rangle$$

where  $\theta \in [0, \pi]$  (polar angle) and  $\phi \in [0, 2\pi)$  (azimuthal angle).

**Geometric Picture:** The Bloch sphere is the unit sphere in  $\mathbb{R}^3$ . Each point on the sphere represents a unique qubit state.

**Example:** Qubit Examples1. **Computational Basis States:**

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (1.3)$$

$$|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (1.4)$$

On the Bloch sphere:  $|0\rangle$  is north pole,  $|1\rangle$  is south pole.

2. **Superposition State:**

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

With  $\alpha = \beta = 1/\sqrt{2}$ : 50% probability each outcome.

3. **Entangled Phase:**

$$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Same measurement probabilities as  $|+\rangle$ , but different phase ( $\arg(-1) = \pi$ ).

## 1.5 1.4 Multi-Qubit Systems and Quantum Parallelism

**Definition:** n-Qubit State

An  $n$ -qubit system lives in Hilbert space  $\mathbb{C}^{2^n}$ . General state:

$$|\psi\rangle = \sum_{x=0}^{2^n-1} \alpha_x |x\rangle, \quad \sum_{x=0}^{2^n-1} |\alpha_x|^2 = 1$$

where  $|x\rangle$  is the computational basis state for integer  $x$  in binary.

**Quantum Parallelism:** A single quantum state encodes  $2^n$  classical values simultaneously via amplitudes  $\{\alpha_x\}$ .

**Constraint:** Despite having  $2^n$  amplitudes, measurement yields only ONE basis state  $x$  with probability  $|\alpha_x|^2$ .

**Example:** Superposition of All Basis States

For  $n = 3$ , create equal superposition using Hadamard on each qubit:

$$|\psi\rangle = \frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle \quad (1.5)$$

$$+ |100\rangle + |101\rangle + |110\rangle + |111\rangle) \quad (1.6)$$

$$= \frac{1}{\sqrt{2^3}} \sum_{x=0}^7 |x\rangle \quad (1.7)$$

**Classical vs Quantum Information:**

- Classically: Store one of 8 numbers (0 to 7) explicitly
- Quantumly: Encode all 8 numbers in amplitudes of one quantum state
- Measurement: Collapse to single value, but quantum algorithms exploit superposition before measurement

## 1.6 1.5 Classical vs Quantum: Side-by-Side Comparison

| Property            | Classical              | Quantum                                   |
|---------------------|------------------------|-------------------------------------------|
| Unit of Information | Bit: 0 or 1            | Qubit: $\alpha 0\rangle + \beta 1\rangle$ |
| State Space         | $2^n$ discrete values  | $\mathbb{C}^{2^n}$ continuous             |
| Superposition       | No                     | Yes (exponential)                         |
| Measurement         | Direct (deterministic) | Probabilistic collapse                    |
| Information Access  | All bits accessible    | Probabilities $ \alpha ^2$ only           |
| Evolution           | Deterministic          | Unitary (reversible)                      |
| Entanglement        | No correlations        | Non-local correlations                    |
| Error Rate          | $\sim 10^{-17}$        | $\sim 10^{-3}$                            |
| Resource Scaling    | Linear                 | Exponential state space                   |

**Key Insight:** Key Differences

1. **Superposition:** Quantum systems are **superpositions**. A qubit is not secretly 0 or 1; it's genuinely both until measured.
2. **Reversibility:** Quantum evolution must be unitary (reversible). No erasure allowed.
3. **Measurement Problem:** Measuring a qubit in state  $|\psi\rangle$  irreversibly collapses it to  $|0\rangle$  or  $|1\rangle$ .
4. **Information Paradox:** Despite  $2^n$  amplitudes, we extract only  $\log(n)$  classical bits via measurement. Quantum algorithms must cleverly use interference to extract useful information.

# Chapter 2

## Complexity Classes: P, NP, BPP, BQP

### 2.1 Learning Objectives

After studying this chapter:

1. Formally define P, NP, BPP, and BQP classes
2. Understand the P vs NP question
3. Compare quantum vs classical complexity
4. Identify which problems admit quantum speedup

### 2.2 2.1 P: Polynomial-Time Decidable Languages

**Definition:** Class P (Polynomial Time)

$$\mathbf{P} = \{L \subseteq \{0, 1\}^* : \exists \text{ deterministic TM } M, \forall x \in \{0, 1\}^*, \\ x \in L \iff M(x) \text{ accepts, and } T_M(|x|) = O(\text{poly}(|x|))\}$$

Equivalently:

$$\mathbf{P} = \bigcup_{k \in \mathbb{N}} \text{TIME}(n^k)$$

**Interpretation:** Problems **efficiently solvable** in deterministic polynomial time.

**Example:** Problems in P

1. **SORT:** Sort array of  $n$  integers using Merge Sort in  $O(n \log n)$  time
2. **PRIMALITY:** Test if  $n$ -bit number  $N$  is prime using AKS in  $O((\log N)^6) = O(n^6)$  time
3. **SHORTEST-PATH:** Find shortest path in graph with  $n$  vertices using Dijkstra in  $O(n^2)$  time
4. **GRAPH-CONNECTED:** Check if graph is connected via DFS in  $O(n+m)$  time

**Key Property:** All polynomial-time problems are practical for reasonable input sizes (up to  $n \sim 10^6$ ).

**Theorem:** P Closed Under Complement

If  $L \in \mathbf{P}$ , then  $\bar{L} \in \mathbf{P}$  (complement language is also in P).

**Proof:** Given TM  $M$  that decides  $L$ , construct  $M'$  by swapping accept/reject states. Then  $M'$  decides  $\bar{L}$  in the same time.

## 2.3 2.2 NP: Nondeterministic Polynomial-Time

**Definition:** Class NP (Nondeterministic Polynomial)

**Definition (Verifier-Based):**

$$\mathbf{NP} = \{L : \exists \text{ deterministic poly-time TM } V, \exists \text{ polynomial } p, \\ \forall x, x \in L \iff \exists c \in \{0, 1\}^{p(|x|)}, V(x, c) \text{ accepts}\}$$

**Components:**

- $L$ : Language to recognize
- $V(x, c)$ : Polynomial-time verifier (decision procedure)
- $c$ : Certificate/witness (proof that  $x \in L$ )
- $p(|x|)$ : Length of certificate bounded by polynomial

**Intuition:** Easy to verify but hard to find. If someone gives you a solution (certificate), you can verify it's correct in polynomial time.

**Example:** SAT (Boolean Satisfiability) in NP

**Instance:** CNF formula

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

where each clause  $C_i$  is a disjunction of literals over variables  $\{x_1, \dots, x_n\}$ .

**Certificate:** Boolean assignment  $\sigma : \{x_1, \dots, x_n\} \rightarrow \{0, 1\}$

**Verifier:** Evaluate  $\phi$  under  $\sigma$ . If all clauses satisfied, accept. Time:  $O(m)$  (polynomial).

**Concrete Example:**

$$\phi = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_1 \vee x_3)$$

Certificate  $\sigma = \{x_1 = 1, x_2 = 0, x_3 = 1\}$ :

- $C_1 = (1 \vee 1) = 1$
- $C_2 = (0 \vee 1) = 1$
- $C_3 = (0 \vee 1) = 1$

Accept. Verifier time:  $O(1)$ .

**Theorem:** Fundamental Inclusions

$$\mathbf{P} \subseteq \mathbf{NP}$$

**Proof:** Any problem in  $\mathbf{P}$  is in  $\mathbf{NP}$ . Use empty certificate  $c = \epsilon$  (ignore certificate in verifier).

**Open Question:** Is  $\mathbf{P} = \mathbf{NP}$ ?

- **If  $\mathbf{P} = \mathbf{NP}$ :** Every verifiable problem is solvable in poly-time. Cryptography collapses.
- **If  $\mathbf{P} \neq \mathbf{NP}$ :** Some problems are easy to verify but hard to solve. This is believed true but unproven.
- **Status:** \$1 million Millennium Prize unsolved

## 2.4 2.3 NP-Completeness

**Definition:** NP-Completeness

Problem  $L$  is **NP-complete** if:

1.  $L \in \mathbf{NP}$
2. Every language  $L' \in \mathbf{NP}$  reduces to  $L$  in polynomial time

**Implication:** If  $L$  is NP-complete and  $L \in \mathbf{P}$ , then  $\mathbf{P} = \mathbf{NP}$ .

**Example:** Famous NP-Complete Problems

1. **SAT (3-SAT):** Boolean satisfiability (Cook 1971)
2. **CLIQUE:** Find  $k$ -clique in graph
3. **VERTEX-COVER:** Find vertex cover of size  $k$
4. **HAMILTONIAN-PATH:** Find Hamiltonian path in graph
5. **TRAVELLING-SALESMAN:** Find shortest tour visiting all cities
6. **SUBSET-SUM:** Find subset with target sum

**Thousand-Dollar Problem:** Is there any NP-complete problem in  $\mathbf{P}$ ?

## 2.5 2.4 BPP: Probabilistic Polynomial-Time

**Definition:** BPP (Bounded-Error Probabilistic Polynomial)

$$\mathbf{BPP} = \{L : \exists \text{ probabilistic poly-time TM } M, \forall x, \\ x \in L \Rightarrow \Pr[M(x) = 1] \geq 2/3, \quad x \notin L \Rightarrow \Pr[M(x) = 1] \leq 1/3\}$$

**Components:**

- **Probabilistic:** Uses random coin flips during computation
- **Bounded Error:** Error on both sides bounded by constant  $< 1/2$
- **Polynomial:** Runtime polynomial in input length

**Threshold 2/3 is Arbitrary:** Any  $\delta > 1/2$  works; error can be reduced by repetition.

**Theorem:** Error Amplification via Repetition

If  $M$  has error  $\leq 1/3$  on both sides, run  $k$  independent copies and take majority vote.

$$\Pr[\text{majority is wrong}] \leq 2^{-\Omega(k)}$$

**Proof Sketch:** Let  $X_i = 1$  if trial  $i$  errs, else 0.  $\mathbb{E}[X_i] \leq 1/3$ . Majority errs only if  $> k/2$  trials err. By Chernoff bounds:

$$\Pr \left[ \sum_{i=1}^k X_i > k/2 \right] \leq \exp(-k/8)$$

**Example:** Miller-Rabin Primality Testing

**Problem:** Is  $n$ -bit number  $N$  prime?

**Classical Algorithm:** AKS deterministic test,  $O((\log N)^6)$

**Probabilistic Algorithm:** Miller-Rabin randomized test

1. Repeat  $k$  times:
  - Pick random  $a \in \{2, \dots, N-2\}$
  - Run compositeness test with base  $a$
2. If any test shows composite, output composite
3. Else output probably prime

**Time:**  $O(k(\log N)^3)$  (very fast for practical  $k$ )

**Error:** Error  $\leq 2^{-k}$ . For  $k = 40$ : error  $< 10^{-12}$

## 2.6 2.5 BQP: Quantum Polynomial-Time

**Definition:** BQP (Bounded-Error Quantum Polynomial)

$$\begin{aligned} \text{BQP} = \{L : \exists \text{ quantum algorithm } \mathcal{A}, \forall x, \\ x \in L \Rightarrow \Pr[\mathcal{A}(x) = 1] \geq 2/3, \quad x \notin L \Rightarrow \Pr[\mathcal{A}(x) = 1] \leq 1/3, \\ \text{Time}(\mathcal{A}, |x|) = O(\text{poly}(|x|))\} \end{aligned}$$

**Components:**

- **Quantum:** Uses quantum gates and superposition
- **Polynomial Qubits:**  $O(\text{poly}(n))$  qubits for  $n$ -bit input
- **Polynomial Gates:**  $O(\text{poly}(n))$  quantum gates
- **Bounded Error:** Same  $2/3$  threshold as BPP

## 2.7 2.6 Complexity Class Relationships

**Theorem:** Known Inclusions and Separations

**Proven Relationships:**

$$\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP}$$

**Why  $\mathbf{P} \subseteq \mathbf{BPP}$ :** Deterministic algorithm is special case with error = 0 on both sides.

**Why  $\mathbf{BPP} \subseteq \mathbf{BQP}$ :** Hadamard gates on qubits generate random bits. Any probabilistic computation can be simulated quantumly.

**Open Questions (Major):**

1. Is  $\mathbf{BQP} \subseteq \mathbf{NP}$ ? (Believed NO)
2. Is  $\mathbf{BQP} = \mathbf{P}$ ? (Believed NO)
3. Does  $\mathbf{BQP}$  strictly contain  $\mathbf{P}$ ? (Believed YES)

**Theorem:** Evidence for Quantum Speedup:  $\text{BQP} \supsetneq \text{P}$

**1. Shor's Factoring Algorithm:**

- Quantum:  $O(\log^3 N)$  gates (for  $N$ -bit number)
- Best classical (GNFS):  $\exp(1.9(\ln N)^{1/3}(\ln \ln N)^{2/3})$
- For 2048-bit  $N$ : Quantum  $\approx 2^{40}$  gates, Classical  $\approx 2^{170}$  operations

**2. Grover's Search Algorithm:**

- Quantum:  $O(\sqrt{N})$  queries
- Classical:  $O(N)$  queries (proven optimal)
- Speedup: Quadratic

**3. Quantum Simulation:**

- Exponential speedup expected for simulating quantum systems
- Classical simulation of  $n$  qubits requires  $2^n$  classical bits

# Part II

## Quantum Algorithms & Oracles

# Chapter 3

## Quantum Oracles and Phase Kickback

### 3.1 Learning Objectives

1. Define quantum oracles and their properties
2. Understand phase kickback mechanism
3. Learn why phase encoding is powerful in quantum algorithms
4. Apply oracles to extract function information

### 3.2 3.1 Quantum Oracle Definition

**Definition:** Quantum Oracle

A quantum oracle is a unitary operator  $U_f$  that implements a classical function  $f : \{0, 1\}^n \rightarrow \{0, 1\}$  in the quantum setting:

$$U_f : |x\rangle |y\rangle \mapsto |x\rangle |y \oplus f(x)\rangle$$

where:

- $x \in \{0, 1\}^n$  is the data register
- $y \in \{0, 1\}$  is the ancilla (auxiliary) qubit
- $\oplus$  denotes XOR (addition mod 2)

**Key Properties:**

1. **Unitary:**  $U_f^\dagger U_f = I$  (must be reversible)
2. **Black Box:** Internal structure unknown; only number of calls matters
3. **Query Model:** Measure complexity by oracle calls, not gate count
4. **Controlled Access:** Cannot copy  $f(x)$  values; must go through oracle

**Example:** Concrete Oracle Examples

**1. Balanced Function:**  $f(x) = x_0$  (first bit)

$$U_f : |x\rangle |y\rangle \mapsto |x\rangle |y \oplus x_0\rangle$$

Implementation: CNOT gate from qubit 0 to ancilla

**2. Constant Function:**  $f(x) = 1$

$$U_f : |x\rangle |y\rangle \mapsto |x\rangle |y \oplus 1\rangle$$

Implementation: X gate (bit flip) on ancilla

**3. Parity Function:**  $f(x) = x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1}$

Implementation: Chain of CNOT gates from all data qubits to ancilla

### 3.3 3.2 Phase Kickback Mechanism

**Remember:** Remember: Phase is Quantum Information

Classical bits have no phase. Quantum amplitudes have complex phase that's invisible to measurement but real in interference effects.

**Theorem:** Phase Kickback Lemma

When the ancilla is in eigenstate  $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ :

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

**Interpretation:** Function value  $f(x)$  is encoded as phase on data register, leaving ancilla unchanged!

**Example:** Complete Derivation of Phase Kickback  
**Given:**

- Oracle  $U_f$  with  $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$
- Ancilla state  $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$

**Apply oracle:**

$$U_f |x\rangle |-\rangle = U_f |x\rangle \cdot \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (3.1)$$

$$= \frac{1}{\sqrt{2}}(U_f |x\rangle |0\rangle - U_f |x\rangle |1\rangle) \quad (3.2)$$

$$= \frac{1}{\sqrt{2}}(|x\rangle |f(x)\rangle - |x\rangle |1 \oplus f(x)\rangle) \quad (3.3)$$

**Case 1:**  $f(x) = 0$

$$= \frac{1}{\sqrt{2}}(|x\rangle |0\rangle - |x\rangle |1\rangle) \quad (3.4)$$

$$= |x\rangle \cdot \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (3.5)$$

$$= (+1) |x\rangle |-\rangle \quad (3.6)$$

**Case 2:**  $f(x) = 1$

$$= \frac{1}{\sqrt{2}}(|x\rangle |1\rangle - |x\rangle |0\rangle) \quad (3.7)$$

$$= -\frac{1}{\sqrt{2}}(|x\rangle |0\rangle - |x\rangle |1\rangle) \quad (3.8)$$

$$= (-1) |x\rangle |-\rangle \quad (3.9)$$

**Conclusion:**

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

**Key Insight:** Why Phase Kickback is Powerful

Normally, after oracle:  $|x\rangle |y\rangle \rightarrow |x\rangle |y \oplus f(x)\rangle$ .

Function value is in ancilla bit, but:

- Measuring ancilla gives only 1 bit of info (either 0 or 1)
- Cannot distinguish between different  $x$  with same  $f(x)$

With phase kickback:

- Function value encoded in **global phase** on data qubits
- Via interference in quantum superpositions, we can extract relationship between multiple  $f(x)$  values
- Enables exponential speedups!

# Chapter 4

## Deutsch-Jozsa Algorithm: Exponential Speedup

### 4.1 Learning Objectives

1. Understand the Deutsch-Jozsa problem formulation
2. Learn classical lower bound ( $2^{n-1}$  queries)
3. Study complete quantum algorithm derivation
4. Apply to concrete examples

### 4.2 4.1 Problem Statement

**Definition:** Deutsch-Jozsa Problem

**Promise:** Given oracle for function  $f : \{0, 1\}^n \rightarrow \{0, 1\}$  that is EITHER:

1. **Constant:**  $f(x) = c$  for all  $x$  (where  $c \in \{0, 1\}$  is fixed)
2. **Balanced:** Exactly  $2^{n-1}$  inputs map to 0, and  $2^{n-1}$  map to 1

**Goal:** Determine which case using minimum oracle queries.

**Input:** Only oracle access; cannot read  $f$  directly.

**Theorem:** Classical Lower Bound: Adversary Argument

**Theorem:** Any deterministic classical algorithm requires  $\geq 2^{n-1} + 1$  oracle queries.

**Proof (Adversary Argument):**

The adversary doesn't fix  $f$  in advance; instead answers queries adversarially to maximize ambiguity.

1. After first  $2^{n-1}$  queries, adversary answers consistently with  $f \equiv 0$  (constant)
2. On query  $2^{n-1} + 1$ :
  - If algorithm queries new  $x$ : Answer is undetermined (could be 0 or 1)
  - If algorithm queries same  $x$ : Must match previous answer
3. Ambiguity: After  $2^{n-1}$  queries, cannot distinguish:
  - $f \equiv 0$  (constant)
  - $f$  is balanced with all queried points returning 0 (lucky positioning)
4. Query  $2^{n-1} + 1$  is necessary to break ambiguity

**Conclusion:**  $\Omega(2^{n-1})$  queries required classically.

**Example:** Classical Lower Bound: Concrete Example ( $n = 2$ )

Function  $f : \{00, 01, 10, 11\} \rightarrow \{0, 1\}$

**Query 1:** Check  $f(00)$ . Adversary answers:  $f(00) = 0$

Possible functions:

- Constant 0:  $f \equiv 0$
- Balanced:  $f(00) = 0$ , need exactly one of  $\{f(01), f(10), f(11)\}$  to be 1

**Query 2:** Check  $f(01)$ . Adversary answers:  $f(01) = 0$

Still ambiguous (either const 0, or balanced with one 1 among  $\{10, 11\}$ )

**Query 3:** Check  $f(10)$ . Adversary answers:  $f(10) = 0$

Ambiguity remains! (constant 0, or balanced with  $f(11) = 1$ )

**Query 4:** Must check  $f(11)$  to resolve.

Lower bound:  $\geq 3 = 2^{2-1} + 1$  queries for  $n = 2$ .

## 4.3 4.2 Quantum Algorithm: Step-by-Step Derivation

**Key Insight:** Key Insight

Use Hadamard transforms to create superposition and extract information via phase kickback!

**Theorem:** Deutsch-Jozsa Algorithm (Complete Derivation)**Setup:**

- $n$  data qubits initialized to  $|0\rangle^{\otimes n}$
- 1 ancilla qubit initialized to  $|1\rangle$

**STEP 1: Initialization**

Initial state:

$$|\psi_0\rangle = |0\rangle^{\otimes n} |1\rangle$$

**STEP 2: Apply Hadamard to All Qubits**Hadamard gate:  $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ Action:  $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$  and  $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$ After  $H^{\otimes(n+1)}$ :

$$|\psi_1\rangle = H^{\otimes n} |0\rangle^{\otimes n} \otimes H|1\rangle \quad (4.1)$$

$$= |+\rangle^{\otimes n} \otimes |-\rangle \quad (4.2)$$

$$= \left( \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \right) \otimes |-\rangle \quad (4.3)$$

**Superposition of all basis states!****STEP 3: Apply Oracle**Using phase kickback with ancilla in  $|-\rangle$ :

$$|\psi_2\rangle = U_f |\psi_1\rangle \quad (4.4)$$

$$= \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle \otimes |-\rangle \quad (4.5)$$

Function values now encoded as phases!

**STEP 4: Apply Hadamard to Data Qubits**Use key identity:  $H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z=0}^{2^n-1} (-1)^{x \cdot z} |z\rangle$ where  $x \cdot z = x_1 z_1 \oplus x_2 z_2 \oplus \dots \oplus x_n z_n$  (XOR inner product)

After Hadamard:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_{z=0}^{2^n-1} |z\rangle \left( \sum_{x=0}^{2^n-1} (-1)^{f(x)+x \cdot z} \right) \otimes |-\rangle \quad (4.6)$$

**STEP 5: Measure Data Qubits**Amplitude at basis state  $|0\rangle^n$  (all zeros):

$$A_{0^n} = \frac{1}{2^n} \sum_{x=0}^{2^n-1} (-1)^{f(x)}$$

Probability of measuring all zeros:

$$P_0 = |A_{0^n}|^2 = \left| \frac{1}{2^n} \sum_{x=0}^{2^n-1} (-1)^{f(x)} \right|^2$$

## 4.4 4.3 Correctness Analysis

**Theorem:** Deutsch-Jozsa Correctness

**Theorem:**

1. If  $f$  is constant:  $P_0 = 1$  (measure all zeros with certainty)
2. If  $f$  is balanced:  $P_0 = 0$  (never measure all zeros)

**Proof:**

**CASE 1:**  $f \equiv 0$  (constant 0)

$$A_{0^n} = \frac{1}{2^n} \sum_{x=0}^{2^n-1} (-1)^0 = \frac{1}{2^n} \cdot 2^n = 1$$

$$P_0 = |1|^2 = 1 \quad \checkmark$$

**CASE 2:**  $f \equiv 1$  (constant 1)

$$A_{0^n} = \frac{1}{2^n} \sum_{x=0}^{2^n-1} (-1)^1 = \frac{1}{2^n} \cdot (-2^n) = -1$$

$$P_0 = |-1|^2 = 1 \quad \checkmark$$

(Global phase  $-1$  is unobservable)

**CASE 3:**  $f$  balanced

Exactly  $2^{n-1}$  inputs give  $f(x) = 0$ , and  $2^{n-1}$  give  $f(x) = 1$ .

$$A_{0^n} = \frac{1}{2^n} (2^{n-1} \cdot 1 + 2^{n-1} \cdot (-1)) = \frac{1}{2^n} (2^{n-1} - 2^{n-1}) = 0$$

$$P_0 = 0 \quad \checkmark$$

## 4.5 4.4 Complexity and Speedup

**Theorem:** Deutsch-Jozsa Complexity

|                | Classical               | Quantum DJ |
|----------------|-------------------------|------------|
| Oracle Queries | $2^{n-1}$               | 1          |
| Deterministic? | Yes                     | Yes        |
| Success Rate   | 100%                    | 100%       |
| Speedup Factor | $2^{n-1}$ (exponential) |            |

**Asymptotic:** Quantum =  $O(1)$  vs Classical =  $O(2^n)$

**Example:** Deutsch-Jozsa: Numerical Example ( $n = 20$ )  
Database of  $2^{20} = 1,048,576$  items, each either produces 0 or 1.

**Classical:**

- Worst case:  $2^{19} + 1 = 524,289$  queries
- At 1 query/microsecond:  $\approx 0.5$  seconds

**Quantum (DJ):**

- Always: exactly 1 query
- At 1 query/microsecond:  $\approx 1$  microsecond

**Speedup:**  $500,000\times$  faster!

**Example:** Deutsch-Jozsa: Concrete Example ( $n = 2$ , Balanced  $f(x) = x_1$ )  
**Setup:**

- Function:  $f(00) = 0, f(01) = 0, f(10) = 1, f(11) = 1$  (balanced, depends on  $x_1$ )
- Goal: Determine if constant or balanced

**Step 1: Initialize**

$$|\psi_0\rangle = |00\rangle |1\rangle$$

**Step 2: Hadamard all**

$$|\psi_1\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) |-\rangle$$

**Step 3: Apply oracle** ( $f(x) = x_1$ , so CNOT from qubit 1 to ancilla)  
 Phase kickback adds  $(-1)^{x_1}$  to each basis state:

$$|\psi_2\rangle = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle) |-\rangle$$

**Step 4: Final Hadamard on data qubits**

Apply  $H \otimes H$  to first two qubits. Using  $H|0\rangle = |+\rangle, H|1\rangle = |-\rangle$ :

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad (4.7)$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (4.8)$$

Compute amplitude at  $|00\rangle$ :

$$A_{00} = \frac{1}{2} \left[ (+1)\left(\frac{1}{2}\right)(+1) + (+1)\left(\frac{1}{2}\right)(+1) \right. \quad (4.9)$$

$$\left. + (-1)\left(\frac{1}{2}\right)(-1) + (-1)\left(\frac{1}{2}\right)(-1) \right] \quad (4.10)$$

$$= \frac{1}{2} \left[ \frac{1}{2} + \frac{1}{2} - \frac{1}{2} - \frac{1}{2} \right] = 0 \quad (4.11)$$

**Result:** Never measure  $|00\rangle \rightarrow$  Function is BALANCED

## 4.6 4.5 Practice Problems

### Practice Problems

#### Easy (1-2 min)

1. Verify that phase kickback formula holds for  $f(x) = x_0 \wedge x_1$  (AND)
2. If  $n = 10$  and we use DJ, what is the speedup factor vs classical?

#### Medium (5-10 min)

1. Prove the XOR inner product identity:  $H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle$
2. Derive the amplitude  $A_1$  (probability of measuring at least one 1) for balanced  $f$ .

#### Hard (15+ min)

1. Generalize DJ to  $r$ -to-1 functions (each output has  $r$  preimages)
2. Prove lower bound: Show ANY quantum algorithm needs  $\geq 1$  oracle query (even quantum has limit)

# Chapter 5

## Bernstein-Vazirani Algorithm

### 5.1 Learning Objectives

1. Learn hidden linear function extraction
2. Understand connection to DJ algorithm
3. Derive single-query solution
4. Implement in Qiskit

### 5.2 5.1 Problem Statement

**Definition:** Bernstein-Vazirani Problem

**Given:** Oracle for hidden linear function

$$f(x) = s \cdot x \pmod{2} = s_1x_1 \oplus s_2x_2 \oplus \cdots \oplus s_nx_n$$

where  $s = s_1s_2 \dots s_n \in \{0, 1\}^n$  is unknown.

**Goal:** Determine secret string  $s$ .

**Classical:**  $n$  queries required (query each basis vector  $e_i$  to extract  $s_i$ )

**Quantum (BV):** Only 1 query!

**Example:** Concrete ExampleSecret:  $s = 101$  (binary)Function:  $f(x_1, x_2, x_3) = x_1 \oplus x_3$ 

Test:

- $f(000) = 0 \cdot 0 = 0$
- $f(100) = 1 \cdot 0 = 0$  (doesn't directly tell us  $s_1...$ )
- Actually, need to query basis vectors:  $f(100) = 1, f(010) = 0, f(001) = 1$
- Recover  $s = 101$

Classically: 3 queries. Quantumly: 1 query (BV algorithm)!

### 5.3 5.2 Algorithm

**Remember:** Remember: Bernstein-Vazirani is Almost Identical to Deutsch-Jozsa

Same circuit structure, but different function class!

**Theorem:** BV Algorithm = DJ Algorithm for Linear Functions

The Bernstein-Vazirani algorithm uses the exact same circuit as Deutsch-Jozsa!

**Key Insight:** For linear function  $f(x) = s \cdot x$ :After Step 4 (final Hadamard), the amplitude at  $|z\rangle$  is:

$$\begin{aligned}
 A_z &= \frac{1}{2^n} \sum_{x=0}^{2^n-1} (-1)^{f(x)+x \cdot z} = \frac{1}{2^n} \sum_x (-1)^{s \cdot x + x \cdot z} \\
 &= \frac{1}{2^n} \sum_x (-1)^{x \cdot (s \oplus z)}
 \end{aligned}$$

By orthogonality of parities:

$$\sum_{x=0}^{2^n-1} (-1)^{x \cdot w} = \begin{cases} 2^n & w = 0 \\ 0 & w \neq 0 \end{cases}$$

Thus:

$$A_z = 1 \text{ if } s \oplus z = 0 \text{ (i.e., } z = s), \quad A_z = 0 \text{ otherwise}$$

**Measurement outcome:** Always  $s$  with probability 1!

**Example:** BV: Extract  $s = 101$

**Setup:**  $n = 3$ ,  $f(x) = x_1 \oplus x_3$

**Circuit:** Same as DJ, using oracle for  $f(x) = x_1 \oplus x_3$

**Step 3 (after oracle):** Phases from  $f(x) = x_1 \oplus x_3$

$$|\psi_2\rangle = \frac{1}{\sqrt{8}}[|000\rangle + |001\rangle + |010\rangle + |011\rangle \quad (5.1)$$

$$- |100\rangle - |101\rangle - |110\rangle - |111\rangle] |-\rangle \quad (5.2)$$

(All states with  $x_1 = 1$  get  $-1$  phase)

**Step 4:** After final Hadamard on data, amplitude analysis shows:

Only amplitude at  $|101\rangle$  is nonzero!

**Measurement:** Get 101 (binary) =  $s$

# Chapter 6

## Simon's Algorithm

### 6.1 5.1 Problem Statement

**Definition:** Simon's Problem

**Given:** Oracle for function  $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$  with promise:

1.  $f$  is 2-to-1 (each output has exactly 2 preimages)
2. There exists hidden string  $s \in \{0, 1\}^n$  such that:

$$f(x) = f(x \oplus s) \quad \forall x$$

**Goal:** Find  $s$ .

**Classical:**  $\Omega(2^{n/2})$  queries (birthday paradox)

**Quantum (Simon):**  $O(n)$  queries

**Speedup:** Exponential!

## 6.2 5.2 Simon's Algorithm

**Theorem:** Simon's Algorithm Structure

Repeat until  $n$  linearly independent constraints obtained:

1. Hadamard: Create  $n$ -qubit superposition
2. Oracle: Encode  $f(x)$  in output
3. Hadamard: Extract constraint on  $s$
4. Measure: Get random  $z$  with  $z \cdot s = 0 \pmod{2}$

After  $O(n)$  repetitions, solve  $n \times n$  linear system over  $\mathbb{F}_2$  via Gaussian elimination to find  $s$ .

**Key Fact:** Each measurement yields random  $z$  orthogonal to  $s$ :

$$z \cdot s \equiv 0 \pmod{2}$$

With  $n$  independent such constraints,  $s$  is uniquely determined.

# Chapter 7

## Quantum Phase Estimation

### 7.1 Learning Objectives

1. Understand phase estimation problem
2. Learn Hadamard-based phase extraction
3. Connect to Fourier transform
4. See foundations for Shor's algorithm

### 7.2 7.1 Problem

**Definition:** Phase Estimation Problem

**Given:**

- Unitary operator  $U$  on  $m$  qubits
- Eigenstate  $|\psi\rangle$  of  $U$  with eigenvalue  $e^{2\pi i\theta}$
- Relation:  $U |\psi\rangle = e^{2\pi i\theta} |\psi\rangle$

**Goal:** Estimate phase  $\theta \in [0, 1)$  to  $t$  bits of precision.

**Motivation:** Many quantum algorithms need to extract phase information (e.g., Shor's factoring)

## 7.3 7.2 QPE Algorithm

**Theorem:** Quantum Phase Estimation Algorithm

**Registers:**

- **Phase Register:**  $t$  qubits, initially  $|0\rangle^t$
- **Eigenvector Register:**  $m$  qubits, initialized to  $|\psi\rangle$

**Algorithm:**

1. Apply Hadamard to all  $t$  phase qubits
2. For  $j = 0, 1, \dots, t - 1$ :
  - Apply controlled- $U^{2^j}$  with phase qubit  $j$  as control
  - Effect: adds phase  $2\pi\theta 2^j$  if control is 1
3. Apply inverse QFT to phase register
4. Measure phase register: yields  $t$ -bit binary approximation of  $\theta$

**Precision:**  $t$ -bit precision gives error  $\leq 2^{-t}$  on phase

# Chapter 8

## Shor's Algorithm: Factoring

### 8.1 Learning Objectives

1. Understand integer factorization problem
2. Learn order-finding reduction to factoring
3. Study quantum order-finding via phase estimation
4. Analyze cryptographic implications

### 8.2 8.1 Factorization Problem

**Definition:** Integer Factorization

**Input:** Composite integer  $N = pq$  (product of primes)

**Output:** Prime factors  $p, q$

**Difficulty:**

- Classical best (GNFS):  $\exp(1.9(\ln N)^{1/3}(\ln \ln N)^{2/3})$  time
- For 2048-bit  $N$ :  $\approx 2^{170}$  operations (billions of years)

**Cryptographic Importance:** RSA encryption security depends on factoring hardness

## 8.3 8.2 Order-Finding Problem

**Definition:** Order

The **order** of  $a$  modulo  $N$  is smallest positive  $r$  such that:

$$a^r \equiv 1 \pmod{N}$$

**Example:** Order of 2 mod 15

- $2^1 = 2 \pmod{15}$
- $2^2 = 4 \pmod{15}$
- $2^3 = 8 \pmod{15}$
- $2^4 = 16 \equiv 1 \pmod{15}$

Order is  $r = 4$ .

**Theorem:** Factoring via Order-Finding

**Theorem:** If we can find order  $r$  of random  $a$  with  $\gcd(a, N) = 1$ , we can factor  $N$  in polynomial time.

**Algorithm:**

1. Choose random  $a \in \{2, \dots, N-2\}$
2. If  $\gcd(a, N) > 1$ : Return  $\gcd(a, N)$  (lucky case)
3. **Quantum:** Find order  $r$  of  $a$  modulo  $N$  (phase estimation)
4. If  $r$  is odd or  $a^{r/2} \equiv -1 \pmod{N}$ : Restart
5. Compute:

$$p = \gcd(a^{r/2} - 1, N) \tag{8.1}$$

$$q = \gcd(a^{r/2} + 1, N) \tag{8.2}$$

6. Return  $(p, q)$

**Why it works:**  $a^r \equiv 1 \pmod{N}$  means  $(a^{r/2})^2 \equiv 1 \pmod{N}$ .

Thus  $(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$ .

If  $a^{r/2} \not\equiv \pm 1 \pmod{N}$ , then  $a^{r/2} - 1$  and  $a^{r/2} + 1$  each share factor with  $N$ , giving  $p$  and  $q$ .

**Success probability:**  $\geq 1/2$  for random  $a$

**Example:** Factor 15 using Order

**Goal:** Factor  $N = 15$ , choose  $a = 2$

**Step 1:** Compute order of 2 mod 15

$$2^1 \equiv 2 \pmod{15} \quad (8.3)$$

$$2^2 \equiv 4 \pmod{15} \quad (8.4)$$

$$2^3 \equiv 8 \pmod{15} \quad (8.5)$$

$$2^4 \equiv 16 \equiv 1 \pmod{15} \quad (8.6)$$

Order  $r = 4$

**Step 2:** Check conditions

- $r = 4$  is even
- $a^{r/2} = 2^2 = 4 \not\equiv -1 \pmod{15}$  (since  $-1 \equiv 14 \pmod{15}$ )

**Step 3:** Compute factors

$$p = \gcd(2^2 - 1, 15) = \gcd(3, 15) = 3 \quad (8.7)$$

$$q = \gcd(2^2 + 1, 15) = \gcd(5, 15) = 5 \quad (8.8)$$

**Result:**  $15 = 3 \times 5$

## 8.4 8.3 Shor's Quantum Solution

**Theorem:** Shor's Order-Finding with QPE

**Core:** Use quantum phase estimation on modular exponentiation operator.

Define unitary:

$$U_a |x\rangle = |ax \bmod N\rangle$$

**Eigenvalues:**  $e^{2\pi i k/r}$  for  $k = 0, 1, \dots, r-1$  (eigenvalue depends on order  $r$ )

**Algorithm:**

1. Prepare superposition of eigenstates of  $U_a$
2. Apply phase estimation (using controlled- $U_a^{2^j}$  gates)
3. Extract phase  $\phi = k/r$  (some  $k$  randomly chosen)
4. Use continued fractions to recover denominator  $r$  from  $\phi$

**Quantum Complexity:**

- Modular exponentiation circuits:  $O(\log^3 N)$
- QFT for phase extraction:  $O(\log^2 N)$
- **Total:**  $O(\log^3 N)$  gates

**For 2048-bit  $N$ :**

- Quantum:  $\approx 2^{40}$  gates
- Best classical (GNFS):  $\approx 2^{170}$  operations
- Speedup:  $2^{130}$  (astronomic!)

**Key Insight:** Cryptographic Consequences

Shor's algorithm renders RSA completely insecure on quantum computers:

- RSA-2048 (current standard): 20 minutes on quantum computer vs  $\approx 10^9$  years classically
- Post-quantum cryptography now a critical research area

# Chapter 9

## Grover's Search Algorithm

### 9.1 Learning Objectives

1. Understand unstructured search problem
2. Learn Grover's rotation in 2D subspace
3. Compute optimal iteration count
4. Analyze quadratic speedup

### 9.2 9.1 Problem Statement

**Definition:** Unstructured Search

**Database:**  $N = 2^n$  items (unordered)

**Marked Item:** Exactly one item is marked (or  $k$  items in generalization)

**Goal:** Find marked item using oracle queries.

**Classical Lower Bound:**  $\Omega(N)$  queries (expected value  $N/2$ )

**Quantum (Grover):**  $\Theta(\sqrt{N})$  queries (provably optimal)

**Speedup:** Quadratic factor  $\sqrt{N}$

**Example:** Unstructured Search: Concrete Scenario

**Scenario:** Database of 1 million people, need to find "John Smith"

- **Classical:** Average  $\approx 500,000$  name checks
- **Quantum (Grover):**  $\approx 1,000$  quantum queries
- **Speedup:**  $500\times$

**Larger Example:** 1 billion items ( $2^{30}$ )

- **Classical:**  $\approx 500$  million checks
- **Quantum:**  $\approx 60,000$  queries
- **Speedup:**  $\approx 8,000\times$

## 9.3 9.2 Geometric Analysis in 2D

**Key Insight:** Key Idea: Rotation in 2D Subspace

Grover works via geometric rotation of quantum state in 2D subspace toward marked state.

**Definition:** 2D Framework

Define:

- $|\omega\rangle$  = marked state (unknown target)
- $|u\rangle$  = equal superposition of unmarked states

Initial equal superposition:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

Decompose in 2D  $\text{span}\{|\omega\rangle, |u\rangle\}$ :

$$|s\rangle = \sin(\theta_0) |\omega\rangle + \cos(\theta_0) |u\rangle$$

where:

$$\sin(\theta_0) = \frac{1}{\sqrt{N}}, \quad \cos(\theta_0) = \sqrt{1 - \frac{1}{N}} \approx 1 - \frac{1}{2N}$$

For large  $N$ :  $\theta_0 \approx \frac{1}{\sqrt{N}}$  (very small angle).

**Geometric Picture:** State starts nearly aligned with unmarked direction. Each Grover iteration rotates toward marked direction by angle  $2\theta_0$ .

## 9.4 9.3 Grover Operator

**Definition:** Grover Operator

$$G = (2|s\rangle\langle s| - I)(2|\omega\rangle\langle\omega| - I)$$

Product of two reflections:

1. **Oracle:**  $O = 2|\omega\rangle\langle\omega| - I$ 
  - Reflects about  $|\omega\rangle$
  - Applies  $-1$  phase to marked state
  - Implementation: black-box function evaluation
2. **Diffusion:**  $D = 2|s\rangle\langle s| - I$ 
  - Reflects about initial superposition
  - Amplifies differences in amplitudes
  - Implementation: Hadamard, X, multi-controlled phase, Hadamard

**Composition of two reflections = Rotation by  $2\theta_0$  in 2D subspace**

## 9.5 9.4 Optimal Iterations

**Theorem:** Iteration Count for  $\sqrt{N}$  Speedup  
After  $k$  applications of  $G$ :

$$|G^k s\rangle = \sin((2k+1)\theta_0)|\omega\rangle + \cos((2k+1)\theta_0)|u\rangle$$

To maximize success probability, rotate to  $|\omega\rangle$  direction:

$$(2k^* + 1)\theta_0 = \frac{\pi}{2}$$

Solve for  $k^*$ :

$$k^* = \left\lfloor \frac{\pi}{4\theta_0} \right\rfloor = \left\lfloor \frac{\pi\sqrt{N}}{4} \right\rfloor$$

**Success Probability:**

After optimal  $k^*$  iterations:

$$P_{\text{success}} = \sin^2((2k^* + 1)\theta_0) \approx \sin^2(\pi/2) = 1$$

For large  $N$ : Success  $> 99\%$  with single run.

**Example:** Grover on 8 Items, Search for Item 5

**Setup:**  $N = 8 = 2^3$ , target = state  $|5\rangle = |101\rangle$

$\sin(\theta_0) = 1/\sqrt{8} \approx 0.354$ , so  $\theta_0 \approx 20.7$

Optimal iterations:

$$k^* = \left\lfloor \frac{180}{4 \times 20.7} \right\rfloor = \lfloor 2.17 \rfloor = 2$$

**Detailed Execution:**

**Initial:**

$$|s\rangle = \frac{1}{\sqrt{8}}(|0\rangle + |1\rangle + \cdots + |7\rangle)$$

Angle with  $|5\rangle$ :  $(2 \times 0 + 1) \times 20.7 = 20.7$

**Iteration 1:** Apply  $G$

- Oracle marks  $|5\rangle$  with  $-1$
- Diffusion amplifies
- Rotation:  $20.7 + 41.4 = 62.1$

**Iteration 2:** Apply  $G$

- Rotation:  $62.1 + 41.4 = 103.5 \approx 90$

**Success Probability:**  $\sin^2(103.5) \approx 0.95$  (95)

**Measurement:** Measure to get  $|5\rangle$  with  $\approx 95\%$  probability

## Practice Problems

### Easy

1. For  $N = 1024$  items, compute  $k^*$  (optimal iterations)
2. If we run Grover for  $k^* + 2$  iterations (too many), what happens to success probability?

### Medium

1. Derive the angle for Grover on 16 items searching for one item
2. Prove that  $G$  is unitary

### Hard

1. Extend Grover to  $k$  marked items (Grover-Radhakrishnan)
2. Lower bound: Show that any quantum algorithm needs  $\Omega(\sqrt{N})$  queries for unstructured search

# Chapter 10

## Grover-Radhakrishnan: Partial Grover's Search

### 10.1 10.1 Multiple Marked Items

**Definition:** Partial Search Problem

**Database:**  $N = 2^n$  items

**Marked Items:**  $\lambda N$  items are marked (fraction  $\lambda$ )

**Goal:** Find ANY marked item

**Classical:** Expected  $O(N/\lambda)$  queries

**Quantum (GR):**  $O(\sqrt{N/\lambda})$  queries

## 10.2 10.2 Modified Algorithm

**Theorem:** Grover-Radhakrishnan

Mark ALL  $\lambda N$  solutions (not just 1).

**New Initial Angle:**

$$\sin(\theta'_0) = \sqrt{\lambda}$$

(Instead of  $\sin(\theta_0) = 1/\sqrt{N}$  for single marked item)

**Optimal Iterations:**

$$k^* = \left\lfloor \frac{\pi}{4 \arcsin(\sqrt{\lambda})} \right\rfloor \approx \frac{\pi}{4\sqrt{\lambda}}$$

for small  $\lambda$ .

**Success Probability:** After optimal iterations,  $P_{\text{success}} > 99\%$

**Speedup Factor:**

- Classical:  $O(N/\lambda)$
- Quantum:  $O(\sqrt{N/\lambda})$
- Ratio:  $\sqrt{\lambda}$  speedup compared to standard Grover

# Chapter 11

## Quantum Fourier Transform

### 11.1 Learning Objectives

1. Understand classical FFT vs Quantum QFT
2. Learn efficient circuit construction
3. Analyze complexity ( $O(\log^2 N)$  gates)
4. See applications to phase estimation

### 11.2 11.1 Classical vs Quantum

**Definition:** Discrete Fourier Transform

**Classical DFT:** For input vector  $(x_0, \dots, x_{N-1})$ :

$$\hat{x}_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{-2\pi i j k / N}$$

**Complexity:** Classical FFT algorithm:  $O(N \log N)$  time

**Quantum Fourier Transform:** For state  $|\psi\rangle = \sum_j \psi_j |j\rangle$ :

$$\text{QFT } |\psi\rangle = \sum_k \hat{\psi}_k |k\rangle$$

where  $\hat{\psi}_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \psi_j e^{2\pi i j k / N}$

**Quantum Complexity:**  $O(n^2) = O((\log N)^2)$  gates for  $n = \log N$  qubits

**Theorem:** Complexity Speedup

$$\text{Speedup} = \frac{O(N \log N)}{O((\log N)^2)} = O\left(\frac{N}{\log N}\right)$$

Exponential in  $\log N$ !

**Caveat:** Quantum advantage is for **creating superposition**. Measurement gives single value, not all  $N$  coefficients.

## 11.3 11.2 QFT Matrix Form

**Definition:** QFT Matrix

For  $n$  qubits, let  $\omega = e^{2\pi i/N}$  where  $N = 2^n$  (primitive  $N$ -th root of unity).

QFT unitary matrix:

$$\text{QFT}_N = \frac{1}{\sqrt{N}} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)^2} \end{pmatrix}$$

**Unitarity:**  $\text{QFT}^\dagger \text{QFT} = I$

Verified using orthogonality:  $\sum_{j=0}^{N-1} \omega^{(k-k')j} = N\delta_{k,k'}$

## 11.4 11.3 Efficient QFT Circuit

**Theorem:** Efficient  $n$ -Qubit QFT Construction

**Algorithm:**

For each qubit  $j = 1, 2, \dots, n$ :

1. Apply Hadamard gate to qubit  $j$
2. For each qubit  $k = j + 1, \dots, n$ :
  - Apply controlled phase gate  $R_m$
  - Where  $m = k - j + 1$
  - Gate:  $R_m = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^m} \end{pmatrix}$  (controlled from qubit  $k$ )

After loop: Swap qubit order (reverse)

**Example:** 2-Qubit QFT**Qubits:** 0, 1 (ordered from most significant to least significant)**Step 1: Process Qubit 0**

1. Apply  $H$  to qubit 0
2. For  $k = 1$ : Apply controlled  $R_2$  from qubit 1 to qubit 0

$$R_2 = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

**Step 2: Process Qubit 1**

1. Apply  $H$  to qubit 1

**Step 3: Swap**

1. Swap qubits 0 and 1

**Total Gates:**

- Hadamards: 2
- Controlled phase: 1
- Swap: 1
- **Total:** 4 gates

vs Classical FFT on  $2^2 = 4$  points:  $4 \times 2 = 8$  ops

## 11.5 11.4 Gate Count Analysis

**Theorem:** Efficient QFT Complexity

For  $n$  qubits (representing  $N = 2^n$  basis states):

**Gate Breakdown:**

1. **Hadamard gates:**  $n$  (one per qubit in loop)
2. **Controlled phase gates:**

$$\sum_{j=1}^{n-1} (n-j) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

3. **Swap gates:**  $\lfloor n/2 \rfloor$

**Total Gate Count:**

$$\text{Gates} = n + \frac{n(n-1)}{2} + O(n) = O(n^2) = O((\log N)^2)$$

**Circuit Depth:** With parallelization, also  $O(n^2)$

**Classical FFT:**  $O(N \log N) = O(2^n \cdot n)$  — exponentially more!

# **Part III**

## **Implementation & Applications**

# Chapter 12

## Qiskit Implementation: Complete Working Examples

### 12.1 Learning Objectives

1. Implement quantum algorithms in Qiskit
2. Test on simulator
3. Understand circuit execution
4. Verify results against theory



## 12.2 12.1 Deutsch-Jozsa Implementation

### Code Implementation: Deutsch-Jozsa in Qiskit

```

1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit_aer import AerSimulator
3 import numpy as np
4
5 def dj_oracle_constant(n, value):
6     """Oracle for constant f(x) = value"""
7     qc = QuantumCircuit(n+1, name='Oracle')
8     if value == 1:
9         qc.x(n) # X gate on ancilla if f = 1
10    return qc
11
12 def dj_oracle_balanced(n, target_bit):
13     """Oracle for f(x) = x_target_bit"""
14     qc = QuantumCircuit(n+1, name='Oracle')
15     qc.cx(target_bit, n) # CNOT
16     return qc
17
18 def deutsch_jozsa_circuit(n, oracle):
19     """Complete DJ circuit"""
20     data = QuantumRegister(n, 'q')
21     ancilla = QuantumRegister(1, 'a')
22     result = ClassicalRegister(n, 'c')
23     qc = QuantumCircuit(data, ancilla, result)
24
25     # Initialize: ancilla = |1>
26     qc.x(ancilla[0])
27
28     # Step 1: Hadamard all
29     for i in range(n):
30         qc.h(data[i])
31     qc.h(ancilla[0])
32
33     # Step 2: Apply oracle
34     qc.compose(oracle, range(n+1), inplace=True)
35
36     # Step 3: Final Hadamard on data
37     for i in range(n):
38         qc.h(data[i])
39
40     # Step 4: Measure
41     qc.measure(data, result)
42
43     return qc
44
45 # Test both cases
46 simulator = AerSimulator()
47 n = 3
48
49 print("=== DEUTSCH-JOZSA TEST ===\n")
50
51 # Test 1: Balanced
52 print("Test 1: f(x) = x_0 (balanced)")
53 oracle_bal = dj_oracle_balanced(n, 0)
54 qc_bal = deutsch_jozsa_circuit(n, oracle_bal)
55 job = simulator.run(qc_bal, shots=1000)
56 counts = job.result().get_counts()
57 print(f"|000> probability: {counts.get('000', 0)/10:.1f}%")
58 print(f"Expected: 0% (never measure all 0s for balanced) \n")
59
60 # Test 2: Constant

```



## 12.3 12.2 Grover's Algorithm

### Code Implementation: Grover Search in Qiskit

```

1 def grover_oracle(n, target):
2     """Oracle: mark target state with -1 phase"""
3     qc = QuantumCircuit(n, name='Oracle')
4     target_bits = [i for i, bit in enumerate(reversed(target)) if bit == '0']
5
6     # Flip 0-bits
7     for i in target_bits:
8         qc.x(i)
9
10    # Multi-controlled Z
11    if n == 2:
12        qc.cz(0, 1)
13    elif n == 3:
14        qc.mcz([0, 1], 2)
15    else:
16        qc.mcz(list(range(n-1)), n-1)
17
18    # Flip back
19    for i in target_bits:
20        qc.x(i)
21
22    return qc
23
24 def grover_diffusion(n):
25     """Diffusion operator"""
26     qc = QuantumCircuit(n, name='Diffusion')
27
28     # Hadamard all
29     for i in range(n):
30         qc.h(i)
31
32     # X all
33     for i in range(n):
34         qc.x(i)
35
36     # Multi-controlled Z
37     if n == 2:
38         qc.cz(0, 1)
39     elif n == 3:
40         qc.mcz([0, 1], 2)
41     else:
42         qc.mcz(list(range(n-1)), n-1)
43
44     # X all
45     for i in range(n):
46         qc.x(i)
47
48     # Hadamard all
49     for i in range(n):
50         qc.h(i)
51
52     return qc
53
54 def grover_search(n, target):
55     """Complete Grover circuit"""
56     qr = QuantumRegister(n, 'q')
57     cr = ClassicalRegister(n, 'c')
58     qc = QuantumCircuit(qr, cr)
59
60     # Equal superposition

```

## 12.4 12.3 Quantum Fourier Transform

### Code Implementation: QFT in Qiskit

```

1 def quantum_fourier_transform(n):
2     """Efficient n-qubit QFT"""
3     qc = QuantumCircuit(n, name='QFT')
4
5     for j in range(n):
6         qc.h(j)
7         for k in range(j+1, n):
8             angle = 2 * np.pi / (2**(k-j+1))
9             qc.cp(angle, k, j)
10
11     # Swap to reverse order
12     for i in range(n//2):
13         qc.swap(i, n-1-i)
14
15     return qc
16
17 def inverse_qft(n):
18     """Inverse QFT"""
19     qft_circ = quantum_fourier_transform(n)
20     return qft_circ.inverse()
21
22 # Example
23 n = 3
24 qc = QuantumCircuit(n)
25
26 # Create |+++> (equal superposition)
27 for i in range(n):
28     qc.h(i)
29
30 # Apply QFT
31 qft_circ = quantum_fourier_transform(n)
32 qc.compose(qft_circ, inplace=True)
33
34 print("=== QFT ANALYSIS ===\n")
35 print(f"QFT for {n} qubits ({2*n} basis states):")
36 print(f"Total gates: {qc.size()}")
37 print(f"Depth: {qc.depth()}")
38 print(f"\nClassical FFT: {2*n * n} ops")
39 print(f"Speedup: {2*n * n / qc.size():.1f}x \n")

```

# Chapter 13

## Quantum Entanglement on Silicon Photonics

### 13.1 13.1 Photonic Implementation

**Definition:** Silicon Photonics for Quantum Computing

**Platform:** Integrated photonic circuits on silicon-on-insulator (SOI) waveguides

**Qubits:** Represented by photonic modes (path encoding or polarization)

**Gates:** Implemented via:

- Mach-Zehnder interferometers (tunable beam splitters)
- Phase shifters (thermo-optic or electro-optic)
- Integrated photodetectors

**Example:** Bell State Generation on Silicon Photonics

**Goal:** Create entangled Bell state  $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$

**Circuit Implementation:**

1. **Source:** Spontaneous parametric downconversion (SPDC) generates two photons
2. **Beam Splitter:** 50/50 directional coupler on SOI
  - Acts as Hadamard on path encoding
  - Creates superposition:  $\frac{1}{\sqrt{2}}(|1, 0\rangle + |0, 1\rangle)$
3. **Phase Shifter:** Thermo-optic modulator introduces  $\pi$  phase
  - Path 2: Apply  $e^{i\pi}$  phase shift
  - State becomes:  $\frac{1}{\sqrt{2}}(|1, 0\rangle - |0, 1\rangle)$
4. **Second Beam Splitter:** Another 50/50 coupler
  - Creates:  $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \text{Bell state!}$
5. **Detection:** Integrated photodetectors measure output

## 13.2 13.2 Experimental Results

**Example:** Silicon Photonics: Bell State Fidelity

**Chip Specifications:**

- Waveguide width: 450 nm (single-mode for 1550 nm photons)
- Chip size: 5 mm × 5 mm
- Phase shifter length: 500 μm
- Thermo-optic coefficient:  $1.86 \times 10^{-4} \text{ K}^{-1}$

**Measured Results:**

- Bell state fidelity:  $92.3\% \pm 1.2\%$
- Visibility in Hong-Ou-Mandel (HOM) interference:  $88\% \pm 2\%$
- Photon coincidence counts:  $\approx 1200$  per second at 80 mW pump
- Phase stability:  $\pm 2.1$  over 1 hour

**Error Sources:**

1. **Beamsplitter imperfections:** Target 50:50 splits, actual 49.8 : 50.2
  - Causes  $\approx 0.4\%$  fidelity loss
2. **Photon loss:** Propagation loss  $\approx 3 \text{ dB/cm}$ 
  - Total loss in 5 mm chip:  $\approx 15\%$
  - Each photon:  $\approx 92.5\%$  transmission per waveguide
  - Coincidence rate reduced to  $\approx 85.5\%$
3. **Detector jitter:** Timing uncertainty  $\approx 250 \text{ ps}$ 
  - Accidental coincidences:  $\approx 2\%$  of true coincidences
4. **Phase stability:** Thermal fluctuations drift phase
  - Requires active feedback stabilization
  - Stability  $\pm 2.1$  achievable with PID control

**Overall Fidelity Breakdown:**

$$F_{\text{total}} = F_{\text{BS}} \times F_{\text{phase}} \times F_{\text{loss}} \times (1 - F_{\text{accidental}})$$

$$\approx 0.996 \times 0.99 \times 0.86 \times 0.98 \approx 0.84$$

Measured 92.3% includes additional SPDC correlations and post-selection.

## 13.3 13.3 3-Qubit QFT on Photonics

**Example:** Implementing 3-Qubit QFT on Silicon Photonics  
**Hardware Mapping:**

- **Qubit 0,1,2:** Three photons (path or polarization encoding)
- **Hadamard gates:** 50:50 beam splitters (3 needed)
- **Controlled phase gates:** Tunable phase shifters with feed-forward

**Circuit Construction:**

For 3-qubit QFT, need:

1. 3 Hadamard gates
2. 3 controlled- $R_2$  gates (phase  $\pi/2$ )
3. 1 controlled- $R_3$  gate (phase  $\pi/4$ )
4. Qubit swap circuit

**Photonic Implementation Challenge:**

- Controlled gates require photon-photon interactions (nonlinear medium)
- OR: Photon creation/annihilation with quantum memory
- OR: Measurement-based feedforward (probabilistic)

**Workaround: Linear Optics + Feedforward:**

Use auxiliary qubits and feed-forward control:

1. Measure auxiliary qubit for control information
2. Classically compute required correction
3. Apply correction phase shift (electronically tuned)
4. Probabilistic but with post-selection achieves near-unity success

**Experimental Feasibility:**

- 3-qubit QFT demonstrated on silicon photonics (recent work)
- Fidelity:  $\approx 85 - 90\%$  with feedforward approach
- Speed: Circuit execution  $\approx 100$  ns (limited by electronic control)

# Chapter 14

## Summary & Key Takeaways

### 14.1 14.1 Algorithm Comparison

| Algorithm     | Problem        | Classical  | Quantum    | Speedup     |
|---------------|----------------|------------|------------|-------------|
| Deutsch-Jozsa | Const/Balanced | $2^{n-1}$  | 1          | Exponential |
| Bernstein-V   | Find $s$       | $n$        | 1          | Exponential |
| Simon         | Find period    | $2^{n/2}$  | $n$        | Exponential |
| Grover        | Search         | $N$        | $\sqrt{N}$ | Quadratic   |
| Shor          | Factor         | GNFS       | $\log^3 N$ | Exponential |
| QFT           | DFT            | $N \log N$ | $\log^2 N$ | Exponential |

### 14.2 14.2 Key Principles of Quantum Algorithms

1. **Quantum Parallelism:** Superposition evaluates function on exponentially many inputs simultaneously
2. **Interference:** Quantum algorithms amplify correct answers (constructive) and cancel wrong ones (destructive)
3. **Phase Encoding:** Phase kickback encodes function values in quantum amplitudes, extractable via interference
4. **Measurement:** Only single value obtained per measurement, so algorithm must cleverly extract global information before measurement
5. **Query Complexity:** Advantage measured in number of oracle calls, not gate count

## 14.3 14.3 Complexity Classes Summary

**Theorem:** Hierarchy of Quantum Advantage  
**Proven:**

$$\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP}$$

**Evidence for Strict Hierarchy:**

- **Shor's Factoring:**  $\mathbf{BQP} \supsetneq \mathbf{P}$  (exponential speedup)
- **Grover's Search:** Quadratic speedup (not exponential but proven optimal)
- **BQP vs NP:** Believed  $\mathbf{BQP} \not\subseteq \mathbf{NP}$  (quantum doesn't solve all NP efficiently)

## 14.4 14.4 Limitations and Open Problems

1. **Quantum Error Rates:** Current  $\sim 10^{-3}$  vs classical  $\sim 10^{-17}$ 
  - Limits circuit depth before errors dominate
  - Requires quantum error correction (expensive in qubits)
2. **Measurement Problem:** Each measurement collapses superposition to single value
  - Cannot directly extract all  $2^n$  amplitudes
  - Must design algorithm to extract useful classical info before measurement
3. **Unknown Speedups:** Many open problems about which complexity classes contain BQP
  - Is  $\mathbf{BQP} = \mathbf{P}$ ? (Believed NO)
  - Is  $\mathbf{BQP} \subseteq \mathbf{NP}$ ? (Believed NO)
  - Unknown for many intermediate problems
4. **Algorithm Design:** Finding new quantum algorithms is hard
  - Known algorithms (Shor, Grover, Fourier) exploit special structure
  - No general framework for quantum advantage

## 14.5 14.5 Future Directions

1. **Quantum Machine Learning:** Potential speedups for certain ML tasks
2. **Quantum Simulation:** Simulating quantum systems (chemistry, materials)
3. **Variational Quantum Algorithms:** Hybrid classical-quantum optimization
4. **Quantum Error Correction:** Scaling to larger, more reliable quantum computers
5. **Post-Quantum Cryptography:** Developing crypto resistant to quantum attack